협업과 책임감, 자동화를 통해 좋은 제품을 만들어가는 백엔드 개발자

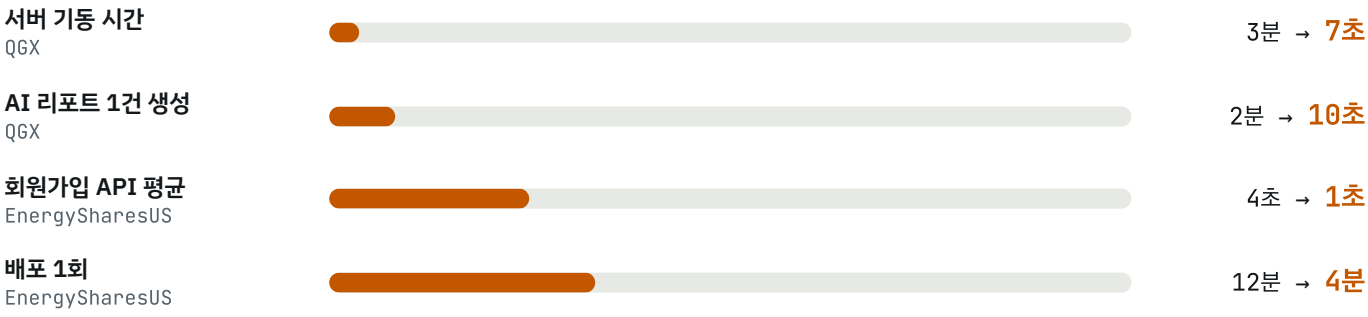
이강일

8년차 백엔드 개발자 · 백엔드 개발 리드

미국 투자 플랫폼의 투자 · 결제 · 에스크로 도메인을 기획 단계부터 만들고 운영했고,
지금은 제조 현장 모니터링 솔루션에서 설비 데이터 입수부터 Kafka 처리, 저장 · 집계 API까지 맡고 있습니다.

현재	디라이트 파트너 · 백엔드 개발 리드 2025.03 ~ 현재
이전	일루미나리안 · 팀원 → 프로젝트 리드 2019.12 ~ 2025.02 / 임플릿 · 팀원 2017.10 ~ 2019.11
도메인	투자 · 결제 · 에스크로 / 제조 설비 데이터
주력	Java · Spring Boot · MySQL · JPA · Kafka · AWS · Docker

IMPACT · 운영 환경에서 잰 전후 숫자



막대 길이 = 수정 후 ÷ 수정 전. 항목마다 수정 전 값을 100%로 둔 비율입니다.

MAIL vcxz2464@gmail.com WEB vcxz2392.github.io

CAREER · 2017.10 ~ 현재

경력 한눈에 보기

2025.03 ~ 현재

디라이트 파트너 백엔드 개발 리드

QGX: 고객사 공장에 서버를 설치하는 제조 현장 모니터링 솔루션 (온프레미스)

- 설비 TCP 입수 → Kafka 메시지 처리 → 저장 · 집계 API 구간 백엔드 리드, 레거시 단일 프로세스를 입수 · 처리 서비스로 분리
- API를 열 수 없는 고객사 기간계(AS400 · MSSQL)를 멀티 데이터소스로 read-only 직접 연결해 매일 입수, AI 리포트 조회를 REST API Tool 호출로 바꿔 2분 → 10초
- SaaS 전환 정부지원사업 기술 리딩, TTA · KACI 클라우드 서비스 시험 통과
- 형상관리 도구를 쓰지 않던 팀에 브랜치 · PR · 리뷰와 Docker 이미지 배포를 도입해 업체 배포 코드 통일, 고객사 문의 창구는 헬프데스크(단독 구축)로 통합

2019.12 ~ 2025.02

일루미나리안 팀원 → 프로젝트 리드 (2024.03~)

EnergySharesUS: 미국 신재생 에너지 크라우드 펀딩 플랫폼 · EnergySharesJP: 일본판

- 기획 단계부터 합류해 회원 · 인증 · 투자 · 결제 · 정산 도메인 백엔드 설계 · 구축과 운영
- 2024.03부터 13명 팀 프로젝트 리드, 백엔드 2명 리드. 리뷰 규칙을 팀과 정하고 검증 · 배포 단계를 나눠 배포 12분 → 4분
- EnergySharesJP: Go · Echo로 회원 · 투자 처리 · 정산 API를 설계 · 구현해 일본 투자자 대상 서비스로 운영

2017.10 ~ 2019.11

임플릿 팀원

케이블 방송사 SI

- 편성 단위를 지역 · 가입자 그룹으로 넓혀 운영자가 그룹별로 편성하게 하고, 셋톱 수집 · 진단 이력을 API로 바로 확인하게 함 (편성 · 배포 운영자 시스템 백엔드와 관리 UI)

컴퓨터공학 학사 (학점은행제) · 2016　수원과학대학교 컴퓨터정보계열 · 2011 ~ 2016　정보처리기사 · 2016.11

CASES · 대표 사례

NO	사례	핵심 결과	쪽
01	한 프로세스에 묶인 입수와 처리를 나눠 실패해도 원본이 남게 디라이트 파트너 · QGX	실패 건만 재처리	03
02	미국 증권 규정을 투자 상태와 검증 규칙으로 일루미나리안 · EnergySharesUS	검증 규칙 12개 카드 투자 오픈	04
03	외부 장애는 사용자에게 번지지 않게, 에러는 쌓이기 전에 일루미나리안 · EnergySharesUS	4초 → 1초 ERROR 로그 알람	05
04	AI 리포트 조회를 SQL 생성에서 API Tool 호출로 디라이트 파트너 · QGX	2분 → 10초	06
05	리뷰 · 배포 흐름을 만들고, 다음 회사로 가져가기 일루미나리안 → 디라이트 파트너	배포 12분 → 4분 업체 배포 코드 통일	07
+	그 밖에 한 일 · 기술 스택 서버 기동 · 기간계 연동 · SaaS 외		08

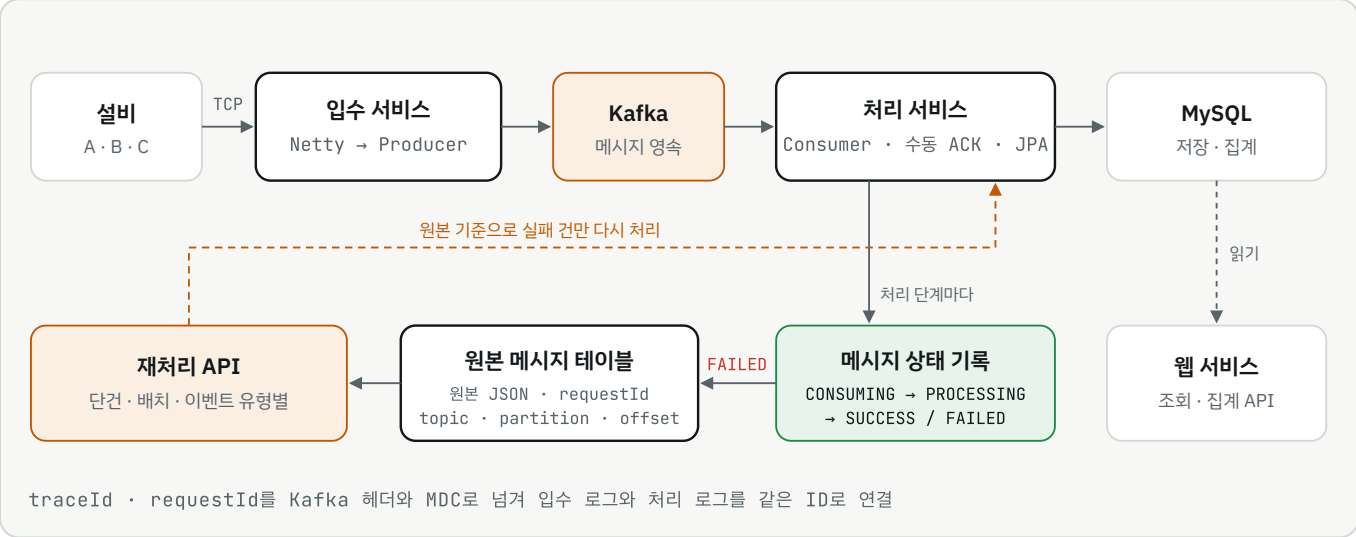
CASE 01 디라이트 파트너 · QGX · 2025.10 ~

한 프로세스에 묶여 있던 입수와 처리를 나눠, 후처리가 실패해도 원본이 남는 구조로

원본 보관

실패 건만 재처리

설비가 TCP로 보낸 값을 같은 프로세스에서 바로 후처리하던 레거시를 입수 · 처리 · 웹으로 나누고, 메시지마다 어디까지 처리됐는지 남겼습니다.



문제	입수 · 처리 · 웹이 한 프로세스에 묶여 장애 격리와 독립 배포가 불가. TCP로 받은 데이터를 같은 프로세스에서 동기로 후처리해, 후처리가 실패하면 원본 기준으로 추적 · 재처리할 경로가 없어 데이터 유실로 이어지는 구조
판단	• 도메인 경계를 지금 긋기엔 이르다고 보고 1차 분리 기준을 기술적 책임(입수 · 처리 · 웹)으로 설정. 독립 배포와 오너십이 필요해질 때마다 하나씩 떼어내는 순서 • 재처리에는 메시지 영속과 원본 보관이 필요해 in-memory queue 대신 Kafka 채택
조치	• 입수(Netty → Kafka Producer) · 처리(Kafka Consumer → JPA) · 웹으로 분리 • 처리 완료 후 offset을 커밋하도록 수동 ACK 적용, 메시지 상태 기록, 원본 메시지 테이블에 원본 JSON · requestId · offset 보관 • 실패 메시지는 단건 · 배치 · 이벤트 유형별 재처리 API로 다시 처리
결과	**입수 · 처리 분리를 운영에 적용. Kafka에 적재된 메시지는 후처리가 실패해도 원본이 남아, requestId · offset 기준으로 원인을 확인하고 실패 건만 다시 처리**

이 사례에서 가져가는 것

데이터를 다루는 시스템에서는 얼마나 빨리 처리하는지보다 **실패했을 때 무엇이 남는지**를 먼저 정합니다. 원본과 처리 상태가 남아 있으면 원인 확인과 재처리가 정해진 운영 절차가 됩니다.

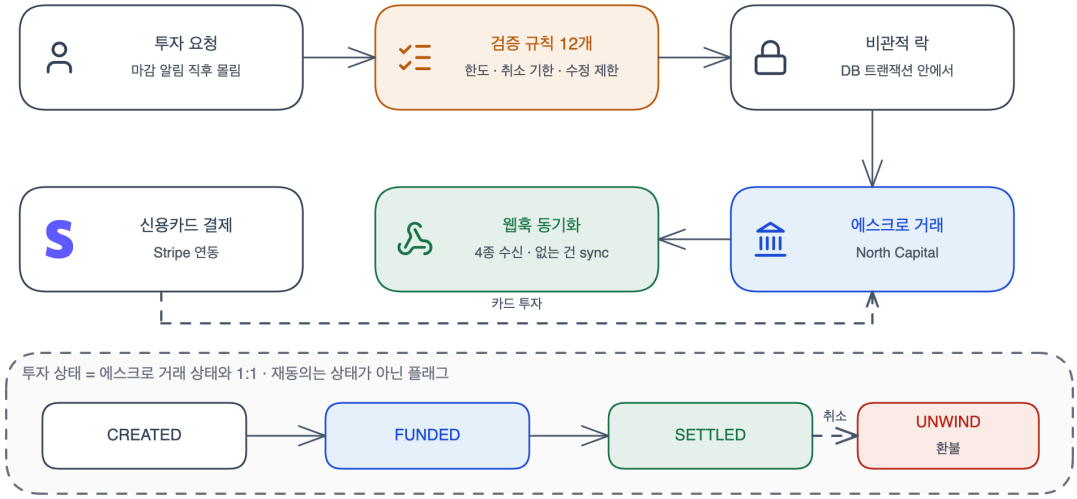
CASE 02 일루미나리안 · EnergySharesUS · 2021.12 ~ 2024

미국 증권 규정을 투자 상태와 검증 규칙으로

검증 규칙 12개

신용카드 투자 오픈 2023 4Q

투자 한도, 취소 기한, 조건 변경 재동의 같은 규정 요구사항을 사람이 기억하는 대신, 투자와 취소 때마다 코드가 같은 규칙으로 검증하게 만들었습니다.



문제	기획 · 컴플라이언스가 정리한 미국 증권 규정(Reg CF · Reg D · Reg A) 요구사항이 투자 한도 · 취소 기한 · 조건 변경 재동의를 요구하고, 에스크로 · KYC/AML · 문서 보관은 제약이 서로 다른 외부 서비스에 의존
판단	• 투자 상태가 에스크로 거래 상태와 1:1로 맞물리는 구조라 상태를 늘리지 않고, 재동의는 플래그로 분리 • 웹훅 부재 · 호출 제한 · 검증 실패 같은 외부 제약은 우리 쪽 상태와 배치로 흡수 • 마감 알림 직후 몰리는 요청은 당시 규모에서 분산 락보다 DB 트랜잭션 안의 비관적 락으로 충분하다고 판단
조치	• 투자 검증 규칙 12개: 결제수단별 한도(카드 $5,000 · ACH $100,000), Reg CF 한도를 투자 시 차감 · 취소 시 복원, 롤링 12개월 1회 수정 제한 • 에스크로를 FundAmerica에서 North Capital로 바꾸며 없는 웹훅은 sync로 대체, Egnyte 호출 제한은 서비스 계정 분산 배치, KYC/AML 실패 건은 결제 보류 후 수동 검토 • 신용카드 투자 결제를 North Capital + Stripe로 연동: 수수료 부담 주체 설정, 진행 중 투자가 있으면 카드 수정 잠금
결과	**규제 한도 · 결제 한도 · 취소 기한을 투자와 취소 때마다 같은 규칙으로 검증. 신용카드 투자 오픈(2023 4분기)**

이 사례에서 가져가는 것

규정이 많은 도메인일수록 상태는 적게 두고, 규칙은 **한곳에서 매번 같은 방식으로** 검증되게 합니다. 외부 서비스의 빈틈은 그쪽이 고쳐 주길 기다리지 않고 우리 쪽 상태와 배치로 메웁니다.

CASE 03 일루미나리안 · EnergySharesUS · 2024

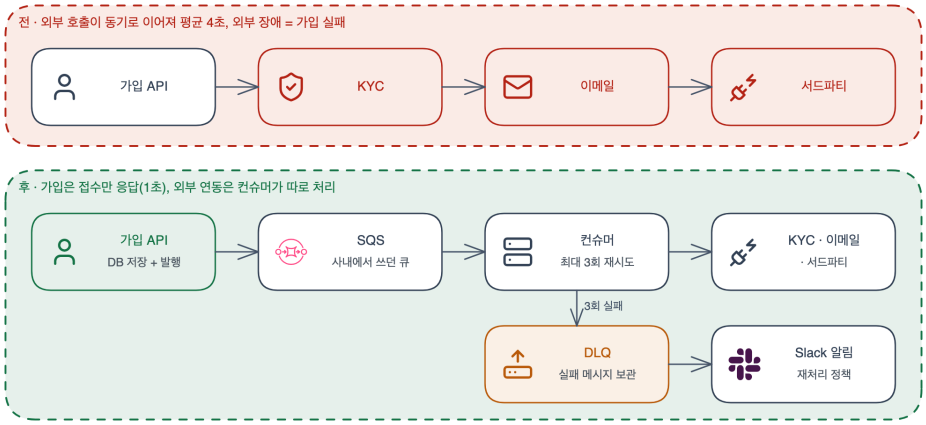
외부 장애는 사용자에게 번지지 않게, 에러는 쌓이기 전에 알게

4초 → 1초
회원가입 API 평균

미국 투자 플랫폼 운영 중 같은 해에 손본 두 가지입니다. 가입 요청이 외부 서비스에 끌려가지 않게 하고, 알람이 놓치던 에러를 잡았습니다.

회원가입 외부 연동 비동기 전환 2024.04

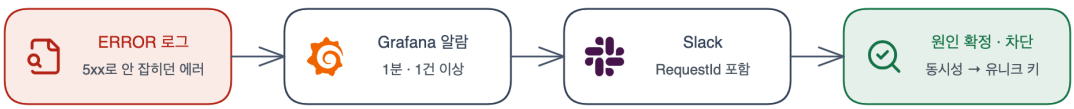
4초 → **1초**



문제	외부 KYC · 이메일 · 서드파티 호출이 동기로 이어져 평균 4초, 외부 지연 시 7초 이상. 외부 장애가 곧 회원가입 실패로 노출
판단	가입 시점에 즉시 필요한 건 접수 완료 여부 하나뿐. 브로커는 사내 메일 서비스가 이미 쓰던 SQS를 재사용
조치	가입 API는 DB 저장과 SQS 발행만, 외부 연동은 컨슈머로 분리. 최대 3회 재시도 후 DLQ로 이동, Slack 알림으로 실패 메시지 가시화
결과	**회원가입 API 평균 4초 → 1초. 외부 KYC · 이메일 처리 완료를 기다리지 않고 가입 접수를 응답해, 외부 장애가 가입 실패로 드러나지 않음**

운영 알람 재설계 2024.09

ERROR 로그 기준



문제	알람이 Nginx 5xx 기준이라 리다이렉션으로 처리되는 에러는 감지 불가. 소셜 가입 중복 계정 에러로 약 400명이 로그인하지 못하는 동안에도 알람이 울리지 않음
조치	• Grafana에서 최근 1분간 ERROR 로그 1건 이상을 알람 조건으로 두고 Slack 발송, 메시지에 RequestId를 넣어 요청 단위로 추적 • 슬로우 쿼리 1초 기준 알람을 개발팀 전용 채널로 분리
결과	**5xx에 안 잡히던 에러와 슬로우 쿼리까지 알람 대상으로 확대. 누적 가입 오류는 RequestId로 추적해 동시성 문제로 확정하고 유니크 키 제약으로 차단. 같은 방식을 사내 다른 프로젝트에도 적용**

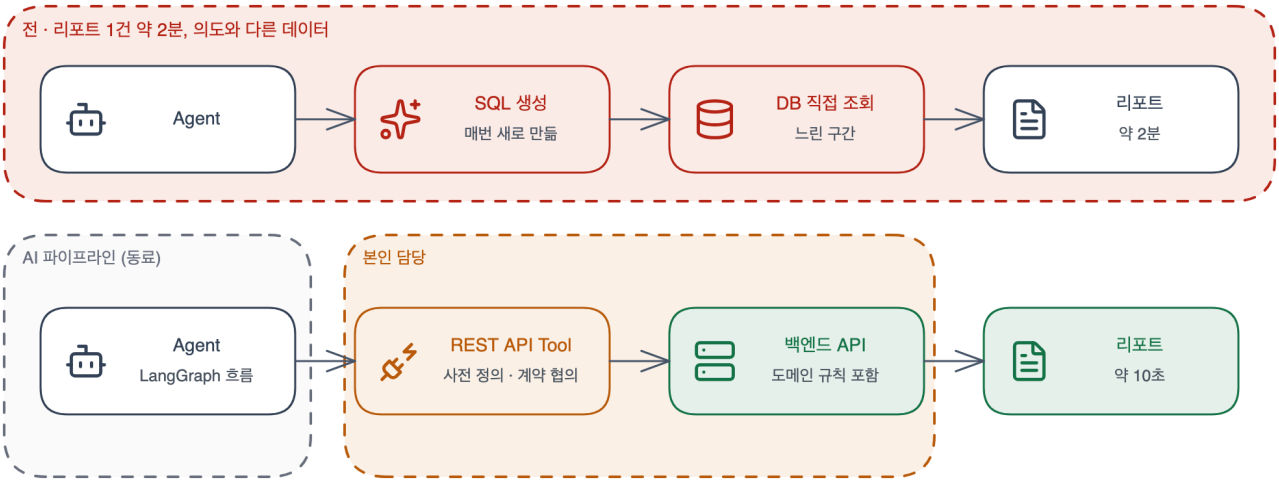
CASE 04 디라이트 파트너 · QGX · 2025.03 ~

AI 리포트 조회를 SQL 생성에서 API Tool 호출로

2분 → 10초
리포트 1건 생성

AI가 매번 SQL을 만들어 DB를 직접 읽게 두는 대신, 도메인 규칙이 이미 들어 있는 백엔드 API를 Tool로 부르게 바꿨습니다.

역할 경계 AI 파이프라인 본체(워크플로우 · 프롬프트)는 AI 담당 동료 몫. 본인은 느린 원인 분석, 조회 방식 전환, Agent가 호출할 API Tool 제공



LangGraph 흐름에서 조회가 일어나는 지점만 Tool 호출로 바꾸고, 나머지 흐름은 그대로 둠

문제	리포트 1건 생성에 약 2분. Agent가 SQL을 생성해 DB를 직접 조회하는 구간이 느리고, 생성된 쿼리가 의도와 다른 데이터를 가져와 원하지 않는 답변이 나오는 상태
판단	매번 쿼리를 생성하게 두는 대신 도메인 규칙이 이미 들어 있는 백엔드 API를 Tool로 호출하게 하면, 느린 조회 구간과 의도와 다른 쿼리가 생기는 경로를 함께 없앨 수 있다고 판단
조치	• LangGraph 흐름에서 조회가 일어나는 지점을 파악해 필요한 조회를 사전 정의된 REST API Tool로 제공 • AI 담당 동료와 Tool 명세, 요청 · 응답 계약 협의
결과	**리포트 1건 생성 2분 → 10초. 해당 리포트 흐름의 SQL 생성 · 직접 조회 구간을 Tool 호출로 대체했고, 이후 리포트도 같은 Tool 계약을 재사용**

이 사례에서 가져가는 것

AI 기능이 느리거나 엉뚱한 답을 낼 때, 모델이나 프롬프트보다 먼저 **데이터를 어디서 어떻게 가져오는지**를 봅니다. 이미 검증된 백엔드 API를 Tool로 열어 주는 것이 백엔드 개발자가 AI 기능에 가장 직접 기여할 수 있는 지점이라고 생각합니다.

CASE 05 일루미나리안 2024 → 디라이트 파트너 2025.03 ~

리뷰 · 배포 흐름을 팀과 만들고, 그 경험을 다음 회사로 가져가기

12분 → 4분

배포 1회

앞 회사에서는 Git을 잘 쓰던 팀의 리뷰와 배포를 다듬었고, 지금 회사에서는 형상관리 도구를 쓰지 않던 팀에 브랜치 · PR · 리뷰와 이미지 배포 절차를 도입했습니다.

일루미나리안

2024 · 13명 팀 프로젝트 리드

리뷰 규칙
1명 이상 승인 후 머지 · 커버리지 80%를 제안해 팀과 정하고 Protection Rule로 강제, 스프린트 · KPT 회고 정례화

검증과 배포 분리
배포 때마다 테스트 점검까지 돌던 흐름을, 검증은 PR(테스트 · OpenAPI · Sonar, 결과는 PR 댓글)에서 끝내고 배포는 배포만 하도록 팀과 함께 재설계
→ 배포 1회 12분 → 4분

배포 실행 단순화
머지 시 이미지 빌드, 배포는 환경 · 태그만 골라 실행
→ 기획 · PM도 협의된 버전을 직접 반영

→

디라이트 파트너

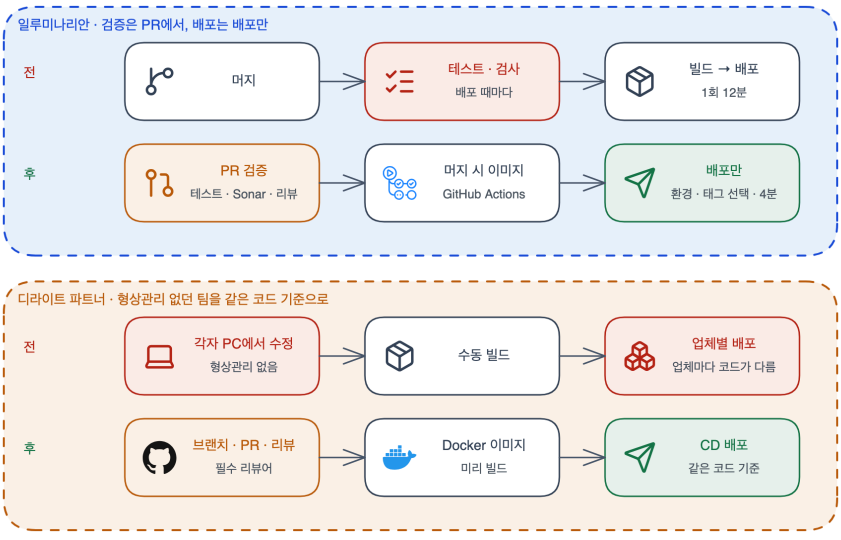
2025.03 ~ · 백엔드 개발 리드

Git · PR · 리뷰 도입
형상관리 없이 각자 PC에서 고쳐 배포하던 팀에 브랜치 전략 · PR 템플릿 · 필수 리뷰어 도입
→ 업체마다 달라지던 코드를 같은 기준으로 배포

Docker 이미지 + CD
빌드 결과물을 매번 수동으로 올리던 배포를 미리 만든 이미지 배포로
→ 배포 시간 단축

AI 개발 도구 팀 도입
온보딩과 리뷰 · 테스트 기준을 저장소 설정으로 통일
→ 대시보드 하나에 몇 달 걸리던 팀원이 1년 뒤 여러 업체를 동시에 맡고, 문서도 자동화

헬프데스크 단독 구축
흩어진 고객사 소통 창구를 기획부터 배포까지 혼자 만든 헬프데스크로 통합
→ 요청 창구 혼선 제거



이 사례에서 가져가는 것

규칙은 혼자 정하지 않고 제안해서 팀과 합의한 뒤 도구로 강제합니다. 사람마다 다르던 기준이 저장소와 파이프라인에 남아야, 사람이 바뀌어도 같은 검증 · 배포 절차를 따르게 된다고 생각합니다.

MORE

그 밖에 한 일

기간계 연동: API 대신 멀티 데이터소스로 직접 입수

디라이트 파트너 · 2025.06 ~

문제 고객사가 API를 열 수 없어 생산계획 · 비가동 · 보전 · 불량 데이터 수집 불가

조치 AS400(JT400) · MSSQL에 멀티 데이터소스로 read-only 직접 연결, 4종 일 배치, 유형 마스터로 재분류. 데이터 성격에 따라 멱등 적재(중복 skip)와 이력 보존형 갱신(is_latest) 구분

결과 API 없이 기간계 데이터를 매일 입수, 설비 실적과 기간계 계획 · 보전 이력을 같은 기준으로 대조

SaaS 전환 정부지원사업 기술 리딩

디라이트 파트너 · 2025.03 ~ 2026.02

문제 고객사마다 서버를 설치하던 구조의 SaaS 전환, PM 부재

조치 일정 · 외부 업체 협의 · 기술 결정 담당, 고객사별 물리 DB 격리, 설치형 배포 Docker화

결과 TTA · KACI 클라우드 서비스 시험 통과로 사업 요건 충족

서버 기동 시간 단축 3분 → 7초

디라이트 파트너 · 2025

문제 네이티브 쿼리가 JPQL로 인식돼 기동 시 전량 파싱 · 검증, 불필요한 빈 로딩까지 겹쳐 기동에 약 3분

조치 해당 쿼리를 Repository 구현체와 JPA Method로 교체, 불필요한 빈 로딩 · 의존성 정리

결과 기동 3분 → 7초

규제 대응 JWT 즉시 차단

일루미나리안 · 2023.04

문제 운영자가 사용자를 차단해도 발급된 JWT는 만료 전까지 유효

조치 차단 대상 jti만 Redis에 저장, TTL을 토큰 만료 시각에 맞춤

결과 차단 직후 요청부터 인증 거부(즉시 로그아웃), 전체 세션을 저장하지 않고 컴플라이언스 요구 충족

감사 로그 구조 재설계

일루미나리안 · 2022.08 ~ 2023.03

문제 엔티티마다 _history 테이블이 늘고, 한 요청에서 함께 바뀐 것을 추적 불가

조치 커스텀 EntityListener로 단일 audit_log, 요청마다 RequestId 발급

결과 운영팀이 한 요청의 변경 이력을 한 번에 조회

스키마 버전 관리 통일 (Flyway)

일루미나리안 · 2023.01 ~

문제 ddl-auto=update와 수동 DDL이 섞여 QA 배포마다 테이블이 어긋남

조치 ddl-auto=none, 모든 변경을 Flyway로, Testcontainers로 매핑 검증

결과 이후 QA · Stage · Production 간 스키마 불일치가 다시 생기지 않음

조회 API 읽기 DB 분리

일루미나리안 · 2024

문제 여러 팀 · 서비스가 같은 DB에 붙어 커넥션이 몰림

결과 MySQL 복제 읽기 · 쓰기 분리, 조회 API를 읽기 DB로 옮겨 쓰기 DB 커넥션 여유 확보

파일 업로드 구조 전환

일루미나리안

문제 모바일 다중 업로드가 API Gateway 10MB 제한 · WAF 룰에 막힘

결과 Cognito 인증 + S3 직접 업로드로 전송 부하를 백엔드에서 빼고 업로드 실패 해소

STACK

주력	**Java · Spring Boot 3 · MySQL 8 · JPA / QueryDSL · Kafka · AWS · Docker**
BACKEND	Spring MVC · Spring Security · OAuth2 · JWT · Netty · MyBatis · Go (Echo · Xorm)
DATA	Redis · Flyway · AS400 / DB2 for i · MSSQL · Oracle
CLOUD	SQS · DLQ · ECS · RDS · S3 · ElastiCache · Cognito · Lambda · NCP · Linux
TEST · CI	JUnit · Mockito · MockMvc · REST Docs · Testcontainers · GitHub Actions · Jenkins · Grafana

CONTACT

협업과 책임감, 자동화를 통해 좋은 제품을 만들어가는 백엔드 개발자

이강일

데이터가 끊기지 않는 구조를 먼저 만들고, 느리거나 숫자가 맞지 않으면 API에서 쿼리, 원본 데이터까지 거슬러 올라가 원인을 확인한 뒤 고칩니다. 팀이 같은 기준으로 일할 수 있게 규칙과 도구를 함께 만드는 일도 계속하고 있습니다.

MAIL	vcxz2464@gmail.com
WEB	vcxz2392.github.io
GITHUB	github.com/vcxz2392

© 2026 이강일